

Building a Simple Graph RAG for Biomedical Questions

Project Objectives

The project aimed to:

- Understand graph database concepts and Neo4j.
- Design a biomedical knowledge graph.
- Query biomedical information using Cypher.
- Generate embeddings for biomedical entities.
- Store embeddings inside Neo4j.
- Build a Graph RAG pipeline for biomedical question answering.
- Connect retrieved graph information with an LLM for natural language responses.

Graph Database Concepts

A graph database stores information as nodes and relationships.

Nodes

Nodes represent entities or objects.

Relationships

Relationships describe connections between nodes.

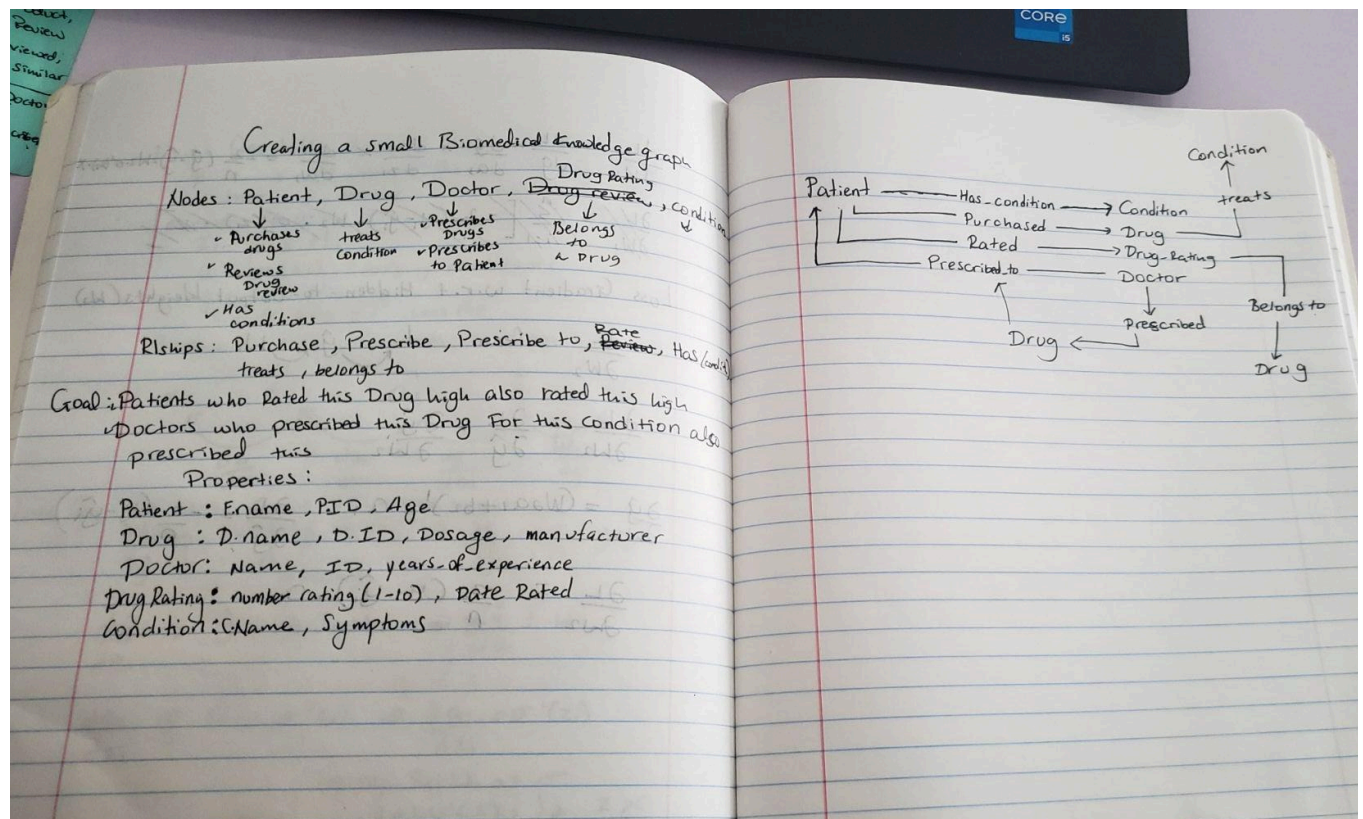
Properties

Properties store additional information about nodes or relationships.

Why Neo4j Was Used

Neo4j was selected because biomedical data is naturally interconnected. For example: A disease can have multiple symptoms, treatments, and affected patients. A graph representation allows these connections to be explored efficiently without complex table joins.

Graph Design



The biomedical knowledge graph was designed using five primary node types: **Patient** (PatientID, Name, Age), **Disease** (DiseaseID, Name, Category), **Symptom** (SymptomID, Name), **Drug** (DrugID, Name, Dosage), and **DrugRating** (RatingID, Rating, DateRated). The graph contains six relationship types that define connections between these entities: **HAS_DISEASE** (Patient $\rightarrow$ Disease), **HAS_SYMPTOM** (Disease $\rightarrow$ Symptom), **TREATS** (Drug $\rightarrow$ Disease), **TAKES** (Patient $\rightarrow$ Drug), **RATED** (Patient $\rightarrow$ DrugRating), and **BELONGS_TO** (DrugRating $\rightarrow$ Drug). These nodes and relationships represent the structure of the biomedical knowledge graph and allow interconnected information about patients, diseases, symptoms, treatments, and drug evaluations to be stored and queried efficiently. Because real patient data contains privacy concerns, synthetic biomedical data was generated for this project.

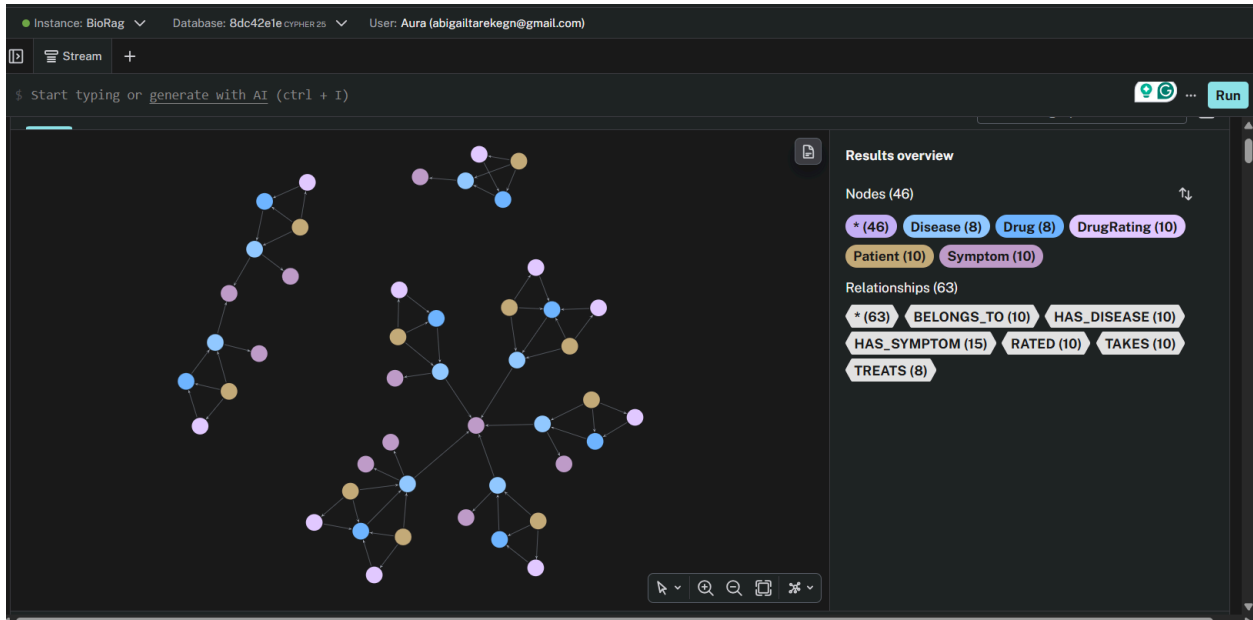
Graph Implementation

The graph was implemented using Cypher query language.

Implementation steps:

1. Created a Neo4j database instance using Neo4j Aura.
2. Created node labels.
3. Inserted synthetic biomedical entities.

4. Created relationships between nodes.
5. Verified node and relationship counts.
6. Removed duplicate relationships.
7. Tested graph queries.



Adding Embeddings to the Knowledge Graph

Embeddings are numerical representations of text that capture semantic meaning. Traditional keyword search requires exact words. For example: The question may be "What medicine helps with high blood sugar?" and the graph contains: High Blood Sugar but a user might ask "Elevated glucose treatment". Embeddings allow the system to understand that Elevated glucose and high blood sugar are related.

Sentence Transformer all-MiniLM-L6-v2 was used in this project. Embeddings were generated for:

- Disease nodes
- Drug nodes
- Symptom nodes

Along with their vector index

SHOW INDEXES;

Table	Raw										
id	name	state	popu	type	entit	labelsOrType	properties	index	owni	lastR	read
2	"disease_embeddings"	"ONLINE"	100.0	"VECTOR"	"NODE"	["Disease"]	["embedding"]	"vector-2026-07"	null	2026-08-07T14:49:38.6050000Z	1
4	"drug_embeddings"	"ONLINE"	100.0	"VECTOR"	"NODE"	["Drug"]	["embedding"]	"vector-2026-07"	null	null	0
1	"index_1b9dcc97"	"ONLINE"	100.0	"LOOKUP"	"RELATIONSHIP"	null	null	"token-lookup-1.0"	null	null	0
0	"index_460996c0"	"ONLINE"	100.0	"LOOKUP"	"NODE"	null	null	"token-lookup-1.0"	null	2026-08-07T14:54:39.0520000Z	172
3	"symptom_embeddings"	"ONLINE"	100.0	"VECTOR"	"NODE"	["Symptom"]	["embedding"]	"vector-2026-07"	null	null	0

Graph RAG Pipeline

The final system combines:

- Knowledge graph
- Embeddings
- Vector search
- LLM generation
- Symptom nodes

Step 1: Semantic Retrieval

The question embedding is compared against symptom embeddings. The system retrieves a symptom: High Blood Sugar

Step 2: Graph Traversal

Neo4j follows relationships: High Blood Sugar => HAS_SYMPTOM=>Type 2 Diabetes
 <=TREATS<=Metformin

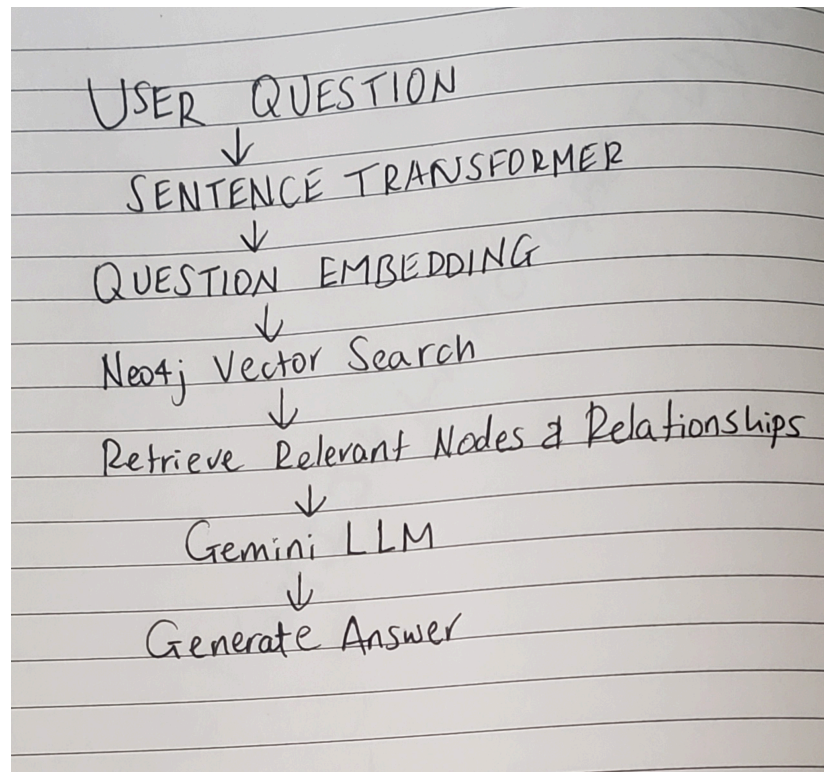
Step 3: Context Generation

The retrieved context is Symptom: High Blood Sugar, Disease: Type 2 Diabetes, Treatment: Metformin, Dosage: 500 mg

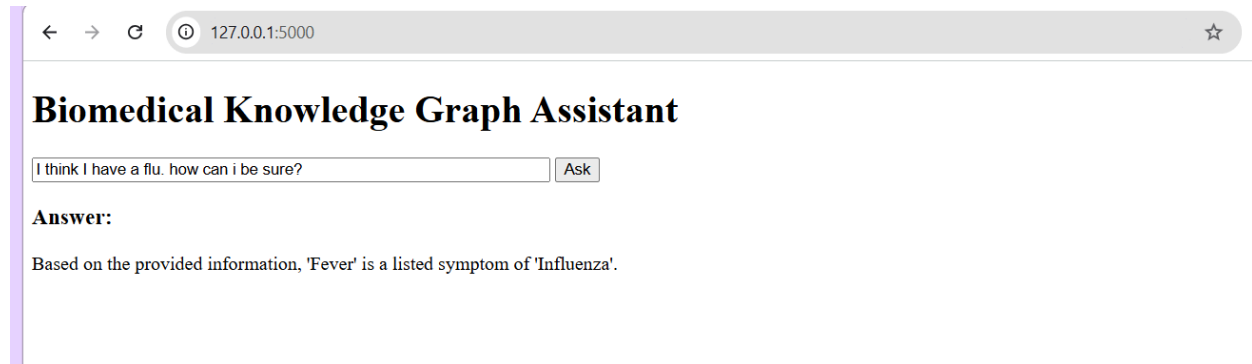
Step 4: LLM Response

Gemini receives the question and retrieves the context.

Generated answer: "Metformin can help with high blood sugar." It is connected to Type 2 Diabetes in the biomedical knowledge graph.



Finally a simple web interface was created using HTML, Flask backend and the Python Graph RAG pipeline.



Challenges Encountered

1. Initially, duplicate relationships were created because the Cypher **CREATE** command always creates new relationships. The problem was solved by: Checking relationship counts and Removing duplicates.
2. Creating a connection between the data in Neo4j and the python code was challenging because of authentication errors. This was solved by creating a new account and starting fresh. Password and userID was then recorded properly the second time which made authentication simpler.
3. The initial Gemini Python package was deprecated. The system was updated to use the newer Google GenAI library for LLM generation. Changed it from Gemini Flash 1.5 to version 2.5
4. Although Cypher is relatively intuitive, writing queries to create relationships, aggregate results, and retrieve connected biomedical information required practice.

Conclusion

This project demonstrated the complete development of a biomedical Graph RAG system using Neo4j.

A synthetic biomedical knowledge graph was created containing:

- 46 nodes
- 63 relationships

The graph successfully represented relationships between patients, diseases, symptoms, drugs, and drug ratings.

The project was extended beyond a traditional knowledge graph by adding:

- Sentence Transformer embeddings

- Neo4j vector indexes
- Semantic retrieval
- Graph-based context expansion
- Gemini LLM generation

The final system can answer biomedical questions by retrieving information from a structured graph instead of relying only on the language model's internal knowledge.

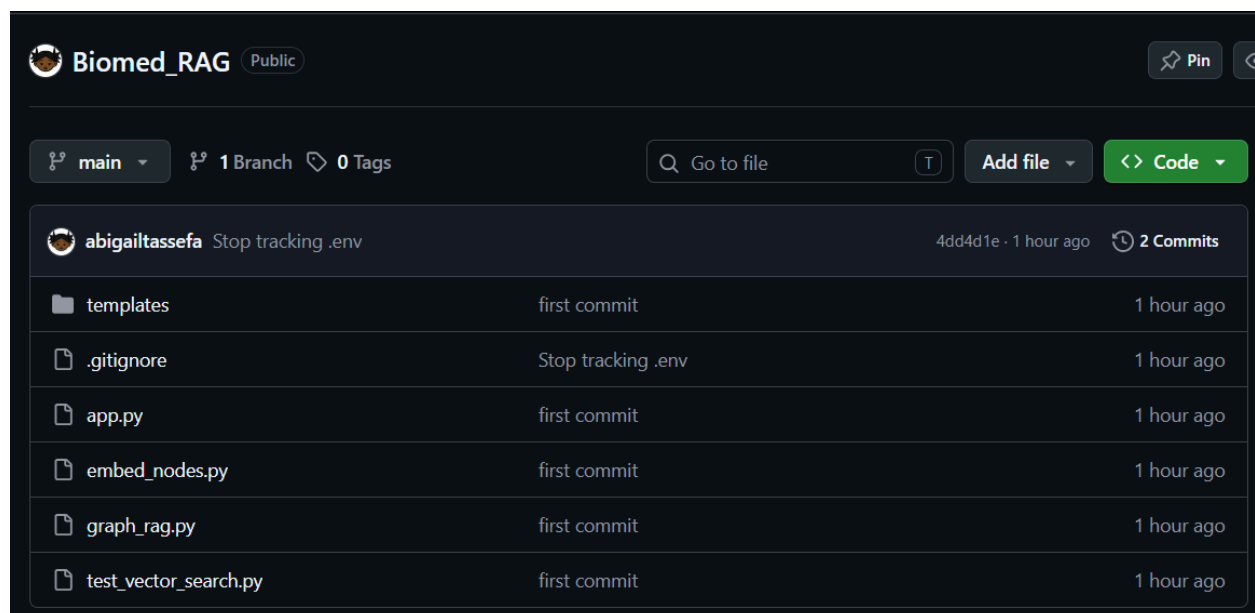
This project provided practical experience with:

- Graph database design
- Cypher querying
- Vector embeddings
- Retrieval-Augmented Generation
- Large Language Model integration

The completed system demonstrates how graph databases and AI models can work together to create intelligent biomedical information retrieval applications.

The whole project is now published as a public github repository.

https://github.com/abigailtassefa/Biomed_RAG.git



Abigail Tarekegn Assefa